

```
// Jarvis - Stage 3 conversation + tools
layer
//
// IMPORTANT: there is no LLM wired into
this build. callModel() below is a
// stub that always reports unavailable.
Every response in this file comes
// from local keyword search or a fixed
system message - never from a
// model - and every message is tagged with
which one produced it, so
// keyword matching is never presented as
if a model were talking.
//
// Wiring up a real model later means
adding an API key and, almost
// certainly, a paid API - per your
guardrails that happens only after
// you explicitly approve it.

(function () {
  'use strict';

  const searchData =
window.JARVIS_SEARCH_DATA;
  const chatLog =
document.getElementById('chat-log');
  const chatForm =
document.getElementById('chat-form');
  const chatInput =
```

```

document.getElementById('chat-input');
const modelBadge =
document.getElementById('model-badge');

    if (!searchData) {
        modelBadge.textContent = 'SEARCH INDEX
MISSING';
        modelBadge.classList.add('offline');
        return;
    }

    // ----- Model interface -----
    // Stage 7: delegates to public/llm.js if
it's loaded and has a
    // configured proxy; otherwise behaves
exactly like the original
    // Stage 3 stub. This function's contract
(query, context) -> { available, ... }
    // hasn't changed shape since Stage 3 -
only what's behind it has.
    async function callModel(query, context)
{
    if (window.JarvisLLM && typeof
window.JarvisLLM.callModel === 'function')
{
        return
window.JarvisLLM.callModel(query, context);
    }
    return { available: false, reason: 'No
LLM is configured in this build.' };
}

    let modelAvailable = false; // updated
after each real call attempt, not just set

```

```

once
  function setSystemState(state) {
    if (typeof
window.JARVIS_ON_STATE_CHANGE ===
'function')
window.JARVIS_ON_STATE_CHANGE(state);
  }

  function renderModelBadge() {
    const badges = [modelBadge,
document.getElementById('model-badge-
2')].filter(Boolean);
    for (const b of badges) {
      if (modelAvailable) {
        b.textContent = 'MODEL ONLINE';
        b.classList.remove('offline');
      } else {
        b.textContent = 'MODEL OFFLINE';
        b.classList.add('offline');
      }
    }
  }
  renderModelBadge();

  // ----- Local search engine
(keyword / TF scoring over cached content)
-----
  function tokenize(str) {
    return (str.toLowerCase().match(/[a-z0-
9]+/g) || []).filter((t) => t.length >= 2);
  }

  const STOPWORDS = new Set([

```

```

    'the', 'a', 'an', 'is', 'are', 'was', 'were', 'be',
    , 'been', 'to', 'of', 'in', 'on', 'for',

    'and', 'or', 'with', 'about', 'what', 'who', 'where',
    , 'when', 'why', 'how', 'me', 'my',

    'you', 'your', 'it', 'this', 'that', 'do', 'does',
    , 'did', 'can', 'could', 'please',

    'find', 'search', 'show', 'tell', 'i', 'we', 'us',
    , 'have', 'has', 'had',
    ]);

    function searchFiles(query, limit = 5) {
        const terms =
        tokenize(query).filter((t) =>
        !STOPWORDS.has(t));
        if (terms.length === 0) return [];

        const results = [];
        for (const file of searchData.files) {
            const nameTokens = tokenize(file.name
            + ' ' + file.category);
            const contentTokens = file.content ?
            tokenize(file.content) : [];
            let score = 0;
            let matchedTerms = new Set();

            for (const term of terms) {
                const nameHits =
                nameTokens.filter((t) => t ===
                term).length;
                const contentHits =
                contentTokens.filter((t) => t ===

```

```

term).length;
    if (nameHits > 0) { score +=
nameHits * 5; matchedTerms.add(term); }
    if (contentHits > 0) { score +=
contentHits * 1; matchedTerms.add(term); }
    }

    if (score > 0) {
        let snippet = null;
        if (file.content) {
            const lower =
file.content.toLowerCase();
            const firstTerm = terms.find((t)
=> lower.includes(t));
            if (firstTerm) {
                const idx =
lower.indexOf(firstTerm);
                const start = Math.max(0, idx -
60);
                const end =
Math.min(file.content.length, idx + 100);
                snippet = (start > 0 ? '...' :
'') + file.content.slice(start, end).trim()
+ (end < file.content.length ? '...' : '');
            }
        }
        results.push({ file, score,
matchedTerms: [...matchedTerms], snippet
});
    }
}

results.sort((a, b) => b.score -
a.score);

```

```

    return results.slice(0, limit);
  }

  // ----- Offline router -----
  // Scores the query against indexed
  // vocabulary (filenames + categories +
  // cached content) to decide "this looks
  // like it's about your files" vs
  // "this looks like open-ended
  // conversation." This is explicit heuristic
  // scoring, shown to the user - never
  // disguised as model reasoning.
  function buildVocabulary() {
    const vocab = new Set();
    for (const file of searchData.files) {
      for (const t of tokenize(file.name))
        vocab.add(t);
      for (const t of
        tokenize(file.category)) vocab.add(t);
    }
    return vocab;
  }
  const vocabulary = buildVocabulary();

  function routeQuery(query) {
    const terms =
      tokenize(query).filter((t) =>
        !STOPWORDS.has(t));
    if (terms.length === 0) return { route:
      'empty', matchedVocab: [] };
    const matchedVocab = terms.filter((t)
      => vocabulary.has(t));
    const vocabRatio = matchedVocab.length
      / terms.length;
  }

```

```

    // Simple, visible threshold: if at
    // least one term matches something in
    // your indexed filenames/categories,
    // treat this as a search request.
    if (matchedVocab.length > 0) {
        return { route: 'search',
        matchedVocab, vocabRatio };
    }
    return { route: 'conversation',
    matchedVocab, vocabRatio };
}

// ----- Chat rendering -----
// speakText (optional): a short plain-
// text version for voice output
// (Stage 4). If omitted, assistant
// messages fall back to a stripped
// version of `text`. This function's
// existing behavior is unchanged
// when Stage 4's voice.js isn't loaded -
// the hook call below is a no-op
// in that case.
function stripHtml(html) {
    const tmp =
document.createElement('div');
    tmp.innerHTML = html;
    return tmp.textContent || tmp.innerText
    || '';
}

function addMessage({ role, kind, text,
meta, speakText }) {
    const wrap =
document.createElement('div');

```

```

wrap.className = `msg msg-${role}`;

if (role === 'assistant') {
  const tag =
document.createElement('div');
  tag.className = `tag tag-${kind}`;
  const tagLabels = { search: 'SEARCH
RESULT', error: 'ERROR', memory: 'MEMORY',
model: 'MODEL' };
  tag.textContent = tagLabels[kind] ||
'SYSTEM · NO MODEL';
  wrap.appendChild(tag);
}

const body =
document.createElement('div');
body.className = 'bubble';
body.innerHTML = text;
wrap.appendChild(body);

if (meta) {
  const metaEl =
document.createElement('div');
  metaEl.className = 'msg-meta';
  metaEl.textContent = meta;
  wrap.appendChild(metaEl);
}

chatLog.appendChild(wrap);
chatLog.scrollTop =
chatLog.scrollHeight;

if (role === 'assistant' && typeof
window.JARVIS_ON_ASSISTANT_MESSAGE ===

```

```

'function') {

window.JARVIS_ON_ASSISTANT_MESSAGE(kind,
speakText || stripHtml(text));
    }
}

function escapeHtml(s) {
    return s.replace(/&<>"/g, (c) => ({
'&': '&','<': '<','>': '>','"': '&quo
t;','"': '&#39;' }[c]));
    }

    // ----- Stage 5: memory commands --
    -----
    // Explicit-only writes: the ONLY way
anything gets stored is the user
    // typing "remember: <fact>" (or
"remember <fact>"). Nothing here or in
    // memory.js infers a fact, summarizes
conversation, or auto-saves
    // anything else. Every branch returns
exactly what happened so it can
    // be shown to the user - never a silent
write.
    function handleMemoryCommand(query) {
        const trimmed = query.trim();

        const rememberMatch =
trimmed.match(/^remember\s*:\s*(.+)$/i) ||
trimmed.match(/^remember\s+(.+)$/i);
        if (rememberMatch) {
            if (!window.JarvisMemory) return {
kind: 'error', text: `Memory isn't

```

```

available in this build (memory.js didn't
load).` };
    const fact = rememberMatch[1].trim();
    const result =
window.JarvisMemory.remember(fact);
    if (!result.ok) return { kind:
'error', text: `Couldn't remember that:
${escapeHtml(result.reason)} ` };
    return {
        kind: 'memory',
        text: `Stored:
"${escapeHtml(result.item.text)}",
        meta: `Browser-local memory ·
${window.JarvisMemory.list().length}
item(s) stored.`,
    };
}

    if (/^(recall|what do you remember|list
memory|show memory)\b/i.test(trimmed)) {
        if (!window.JarvisMemory) return {
kind: 'error', text: `Memory isn't
available in this build.` };
        const items =
window.JarvisMemory.list();
        if (items.length === 0) {
            return { kind: 'memory', text:
`Nothing in memory yet. Say "remember:
<fact>" to store something.` };
        }
        const html = items.map((it, i) =>
            `<div class="result"><div
class="result-name">${i + 1}.
${escapeHtml(it.text)}</div>` +

```

```

        `<div class="result-path">stored
        ${new Date(it.storedAt).toLocaleString()}
        </div></div>`
    ).join('');
    return { kind: 'memory', text: html,
    meta: `Browser-local memory ·
    ${items.length} item(s).` };
    }

    const forgetMatch =
    trimmed.match(/^forget\s*:\s*(.+)$/i) ||
    trimmed.match(/^forget\s+(.+)$/i);
    if (forgetMatch) {
        if (!window.JarvisMemory) return {
        kind: 'error', text: `Memory isn't
        available in this build.` };
        const target = forgetMatch[1].trim();
        const items =
        window.JarvisMemory.list();
        const idx = /\d+$/i.test(target)
        ? parseInt(target, 10) - 1 // 1-
        indexed, matching what "recall" displays
        : items.findIndex((it) =>
        it.text.toLowerCase().includes(target.toLow
        erCase()));
        if (idx < 0 || idx >= items.length) {
            return { kind: 'error', text:
            `Couldn't find a memory item matching
            "${escapeHtml(target)}". Say "recall" to
            see the list.` };
        }
        const result =
        window.JarvisMemory.forget(idx);
        if (!result.ok) return { kind:

```

```

'error', text: `Couldn't forget that:
${escapeHtml(result.reason)}` };
    return {
        kind: 'memory',
        text: `Removed:
"${escapeHtml(result.removed.text)}"`,
        meta: `Browser-local memory ·
${window.JarvisMemory.list().length}
item(s) remaining.` ,
    };
}

    if (/^clear memory$/i.test(trimmed)) {
        if (!window.JarvisMemory) return {
kind: 'error', text: `Memory isn't
available in this build.` };
        const countBefore =
window.JarvisMemory.list().length;
        const result =
window.JarvisMemory.clear();
        if (!result.ok) return { kind:
'error', text: `Couldn't clear memory:
${escapeHtml(result.reason)}` };
        return { kind: 'memory', text:
`Cleared all memory (${countBefore} item(s)
removed).` };
    }

    return null; // not a memory command -
fall through to normal routing
}

async function handleQuery(query) {
    addMessage({ role: 'user', text:

```

```

escapeHtml(query) });

    const memoryResult =
handleMemoryCommand(query);
    if (memoryResult) {
        addMessage({ role: 'assistant', kind:
memoryResult.kind, text: memoryResult.text,
meta: memoryResult.meta });
        if (typeof
window.JARVIS_ON_MEMORY_CHANGE ===
'function')
window.JARVIS_ON_MEMORY_CHANGE();
        return;
    }

    try {
        setSystemState('searching');
        const context = (window.JarvisContext
&& typeof window.JarvisContext.buildContext
=== 'function')
        ?
window.JarvisContext.buildContext(query)
        : { searchResults: [], memory: [],
history: [] };

        const modelResult = await
callModel(query, context);
        modelAvailable =
modelResult.available;
        renderModelBadge();

        if (modelResult.available) {
            const sourceParts = [];
            if (modelResult.sourcesUsed) {

```

```

        if
        (modelResult.sourcesUsed.search)
        sourceParts.push(`${context.searchResults.length} search result(s)`);
        if
        (modelResult.sourcesUsed.memory)
        sourceParts.push(`${context.memory.length} memory item(s)`);
        if (modelResult.sourcesUsed.tools
        && modelResult.sourcesUsed.tools.length) {
            sourceParts.push(`tools used:
        ${modelResult.sourcesUsed.tools.join(',
        ')} `);
        }
    }
    addMessage({
        role: 'assistant', kind: 'model',
        text:
        escapeHtml(modelResult.text),
        meta: sourceParts.length ?
        `Grounded in: ${sourceParts.join('; ')}.` :
        'No indexed search/memory context matched
        this query.',
    });
    if (window.JarvisContext) {

        window.JarvisContext.pushHistory('user',
        query);

        window.JarvisContext.pushHistory('assistant
        ', modelResult.text);
    }
    return;
}

```

```
        // Diagnostic - surfaces exactly why
the model call failed (bad
        // PROXY_URL, token mismatch, missing
Anthropic key on the Worker,
        // network error, etc.) instead of
silently falling back to the
        // offline router with no
explanation. Shown both in the browser
        // console and directly in the chat
UI, as its own small message,
        // so it's visible without needing
dev tools. Does not change the
        // offline router or any of its
existing branches below.
        if (modelResult.reason) {
            console.log('[Jarvis] Model
unavailable:', modelResult.reason);
            addMessage({
                role: 'assistant', kind: 'error',
                text: `Model unavailable:
${escapeHtml(modelResult.reason)}`,
                meta: 'Diagnostic - falling back
to local search below.',
            });
        }

        const routing = routeQuery(query);

        if (routing.route === 'empty') {
            addMessage({
                role: 'assistant', kind: 'error',
                text: `I couldn't find any
searchable words in that.`,
```

```

    });
    return;
  }

  if (routing.route === 'search') {
    const results = searchFiles(query);
    if (results.length === 0) {
      addMessage({
        role: 'assistant', kind:
'search',
        text: `No matches in the
indexed files for:
${escapeHtml(routing.matchedVocab.join(',
'))}.`,
        meta: `Model offline – routed
to local search because these words matched
indexed filenames/categories, but no file
content matched.`,
      });
      return;
    }
    const html = results.map((r) => {
      const snippetHtml = r.snippet ?
`<div class="result-
snippet">${escapeHtml(r.snippet)}</div>` :
'';
      return `<div class="result">
        <div class="result-
name">${escapeHtml(r.file.name)}</div>
        <div class="result-
path">${escapeHtml(r.file.category)} ·
${escapeHtml(r.file.ext)} · score
${r.score}</div>
        ${snippetHtml}

```

```

        </div>`;
    }).join('');
    // Concise version for voice output
    (Stage 4) - the on-screen text
    // above still shows every result
    in full; this is only what gets
    // spoken aloud, so voice doesn't
    read out every snippet verbatim.
    const speakText = results.length
    === 1
        ? `Found 1 match:
    ${results[0].file.name}.`
        : `Found ${results.length}
    matches. Top result:
    ${results[0].file.name}.`;
    addMessage({
        role: 'assistant', kind:
    'search',
        text: html,
        meta: `Model offline – routed to
    local search. Matched indexed terms:
    ${routing.matchedVocab.join(', ')}.\`,
        speakText,
    });
    return;
    }

    // route === 'conversation', but
    there is no model to converse with.
    addMessage({
        role: 'assistant', kind: 'error',
        text: `Model offline, and this
    didn't match anything in your indexed
    files, so I have no grounded way to answer

```

```

it. Try asking about something in your
indexed data – e.g. a filename, client, or
category.` ,
    meta: `Local routing found no
indexed-vocabulary matches for this
query.` ,
    });
    } finally {
        setSystemState('idle');
    }
}

chatForm.addEventListener('submit', (e)
=> {
    e.preventDefault();
    const query = chatInput.value.trim();
    if (!query) return;
    chatInput.value = '';
    handleQuery(query);
});

// ----- Stage 4/7 hook -----
// Exposes handleQuery so voice.js can
feed transcribed speech through the
// exact same router/search path text
input uses - no separate code path,
// no separate behavior. searchFiles is
exposed too (Stage 7) so
// tools.js/context.js can reuse the
exact same search implementation
// instead of duplicating it. Purely
additive; nothing above this line
// that existed before Stage 7 changed
behavior.

```

```

window.JarvisChat = { handleQuery,
searchFiles };

// ----- Startup message -----
// At page load, no call has been
attempted yet, so modelAvailable is
// always false here regardless of
whether a proxy is configured -
// it's only known after a real attempt
(see handleQuery). This
// message stays accurate to that: it
doesn't claim the model is
// connected before it's actually been
verified.
addMessage({
  role: 'assistant', kind: 'error',
  text: `Model status unverified until
the first message. If none is reachable, I
fall back to local search – try a client
name, a category (${[...new
Set(searchData.files.map(f =>
f.category))].join(', ')}), or a filename –
and explicit memory: "remember:
<fact>", "recall", "forget:
<n>", "clear memory".`,
});
})();

```